

TUGAS LAPORAN

Smart Traffic Simulation & Signal Optimalization

Tugas laporan ini disusun untuk memenuhi salah satu persyaratan memperoleh nilai

Mata kuliah Algoritma dan Struktur Data di Program Studi S1 Teknik Elektro

Dosen Pengampu Matakuliah : Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.



Disusun Oleh:

Nama Kelompok:

Amanda Lulu Prasetia : 25051030079

Alwan Fadhilah Rafiyanza : 25051030077

Arwan Adrean M. : 25051030065

Faqih Rizky Saputra : 25051030048

**PRODI TEKNIK ELEKTRO
DEPARTMENT PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
202**

DAFTAR ISI

DAFTAR ISI	1
BAB I	2
PENDAHULUAN	2
BAB II.....	4
PERANCANGAN SISTEM.....	4
BAB III.....	10
IMPLEMENTASI.....	10
BAB IV	15
ANALISIS BIG-O	15
BAB V	21
HASIL EKSPERIMEN	21
BAB VI	24
JAWABAN PERTANYAAN ANALISIS	24
BAB VII.....	30
KESIMPULAN	30
REFERENSI.....	35

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Kemacetan lalu lintas di kota-kota besar di Indonesia menyebabkan kerugian ekonomi triliunan rupiah per tahun. System manajemen lalu lintas cerdas memodelkan jaringan jalan sebagai Graf berbobot (jarak antar persimpangan), mengatur antrian kendaraan dengan Priority Queue(Ambulans selalu prioritas), dan mencari rute optimal dan Dijkstra. Proyek ini membangun simulator lalu lintas berbasis CLI dengan 25 persimpangan dan 40 segmen jalan, mensimulasikan arus kendaraan, penanganan darurat dan optimasi siklus dan lampu lintas.

1.2 Tujuan Sistem

Sistem simulasi lalu lintas pintar ini dibuat untuk mensimulasikan pengaturan lampu lalu lintas secara otomatis berdasarkan kondisi kendaraan pada setiap jalur persimpangan. Sistem bertujuan untuk mengurangi kepadatan kendaraan, mempercepat arus lalu lintas, serta memberikan prioritas kepada kendaraan darurat seperti ambulans. Selain itu, sistem juga dirancang untuk membantu memahami penerapan struktur data dan algoritma pada kasus nyata, khususnya dalam manajemen antrean kendaraan dan pengambilan keputusan lampu lalu lintas secara dinamis.

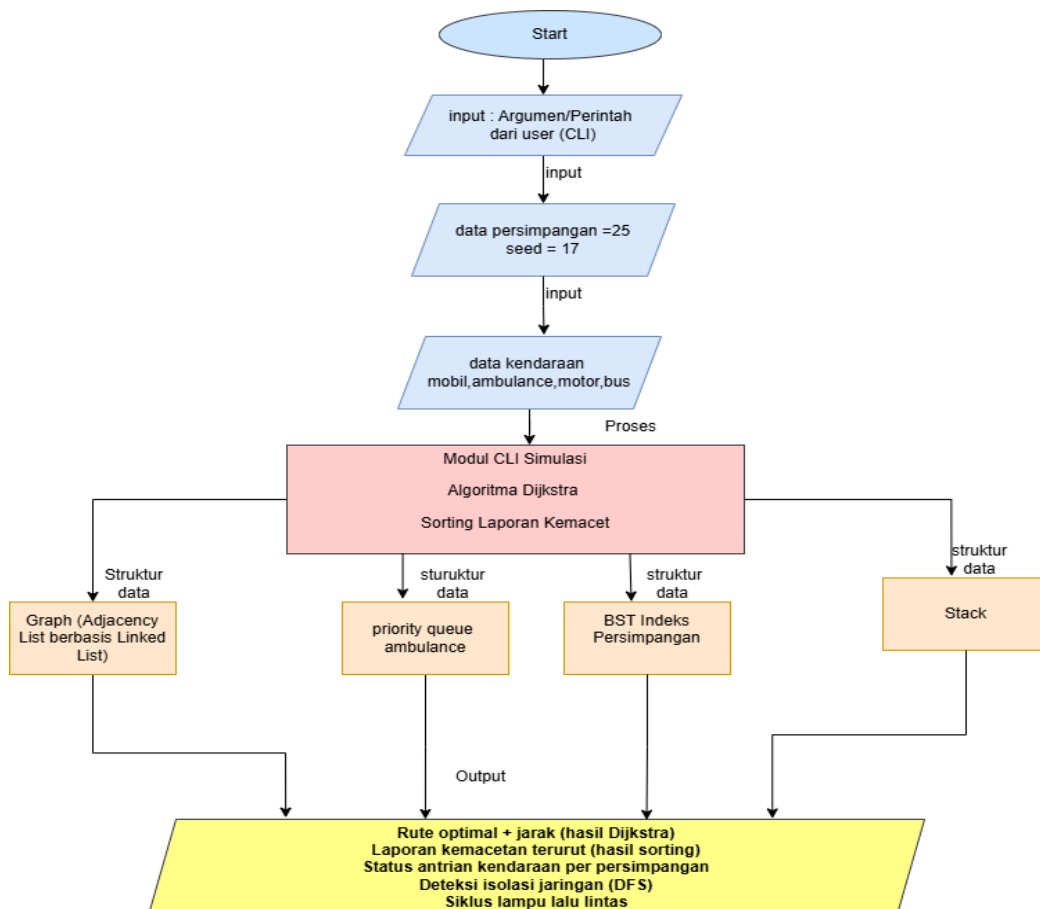
Adapun tujuan utama dalam system sebagai berikut :

1. Mensimulasikan kondisi lampu lalu lintas pada persimpangan
2. Mengatur secara otomatis berdasarkan jumlah kendaraan
3. Memberikan prioritas kepada kendaraan ambulans
4. Menerapkan struktur data, queue, dan algoritma dalam simulasi nyata.
5. Menampilkan data kendaraan masuk, keluar, dan waktu tunggu kendaraan selama simulasi berjalan.

1.3 Batasan Implementasi

Agar system lebih terarah dan sesuai waktu pengerjaan, maka terdapat beberapa batasan implementasi sebagai berikut:

1. Sistem hanya mensimulasikan hanya mencapai batas 25 persimpangan jalan
2. Jenis kendaraan yang digunakan pada simulasi dibatasi menjadi ambulans, bus, mobil, dan motor.
3. Data simulasi pada kendaraan yang keluar masuk pada persimpangan dibatasi hanya mencapai 500 kendaraan
4. System simulasi membatasi jaringan jalan (edge berbobot) hanya 40
5. Untuk pengujian rute dibatasi hanya mencapai 50 pencarian jalur saja
6. Dikasih angka seed (angka random) agar hasil tetap konsisten yaitu seed =17



BAB II

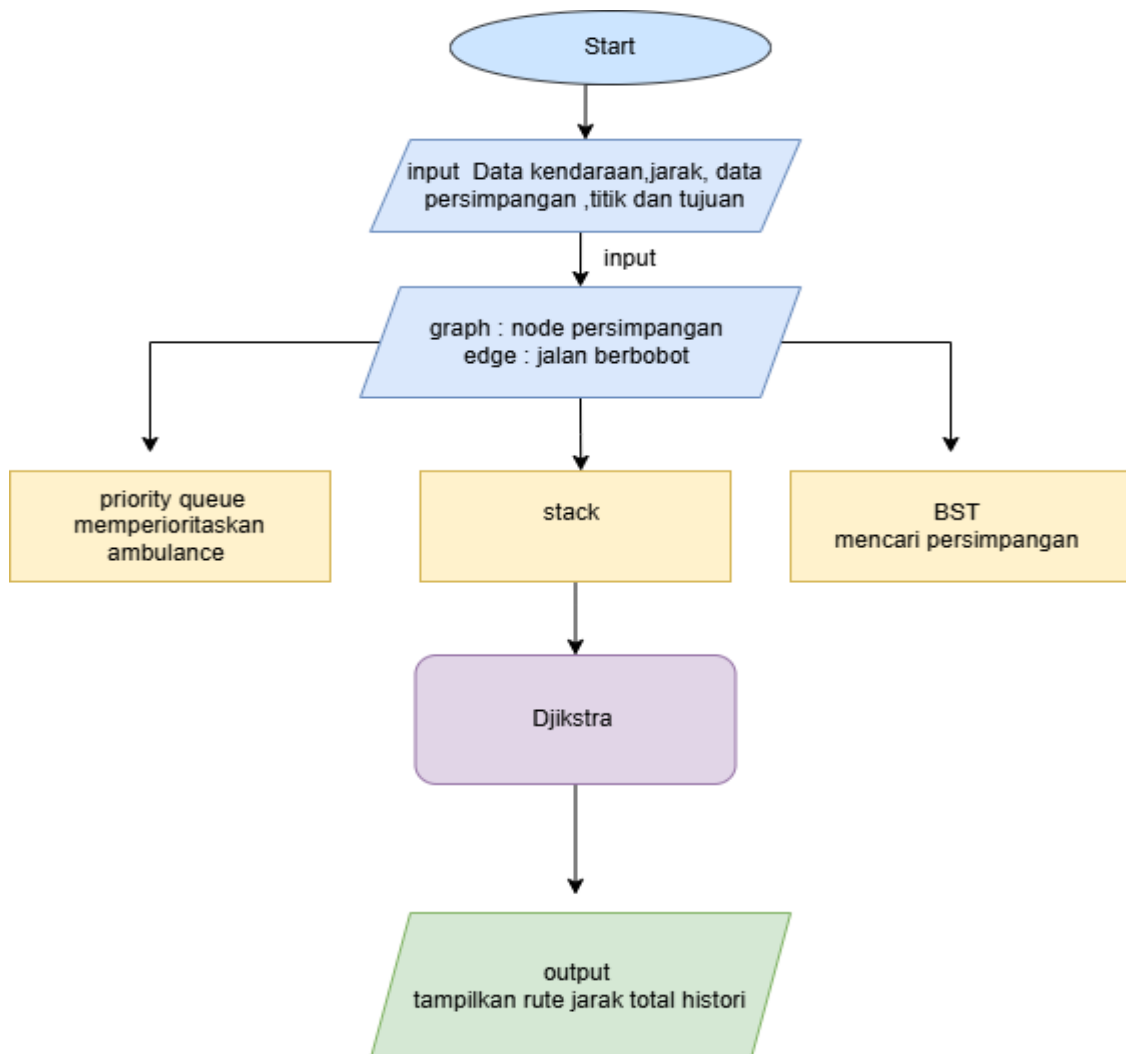
PERANCANGAN SISTEM

2.1 Struktur Data Dan Justifikasi

STRUKTUR DATA	JUSTIFIKASI
Graph (Node + Edge, Directed & Weighted — Dijkstra: $O((V+E) \log V)$)	Graph dipilih karena secara alami merepresentasikan jaringan jalan kota setiap persimpangan dimodelkan sebagai node dan setiap ruas jalan sebagai edge berbobot (jarak/waktu tempuh).
Priority Queue (Min-Heap — Insert & extract-min: $O(\log n)$)	Priority Queue berbasis Min-Heap dipilih karena algoritma Dijkstra mensyaratkan bahwa node dengan akumulasi jarak terkecil selalu diproses terlebih dahulu.
Stack (LIFO — Push & pop: $O(1)$)	Stack dipilih untuk mendukung mekanisme backtracking dan pencatatan riwayat rute selama traversal. Sifat LIFO (Last In, First Out) secara inheren selaras dengan kebutuhan rollback ketika jalur yang ditempuh buntu, sistem cukup melakukan <i>pop</i> untuk kembali ke persimpangan terakhir tanpa harus mengulang dari awal.
BST (Binary Search Tree — Search/insert rata-rata: $O(\log n)$)	BST dipilih untuk mengelola data nama persimpangan secara terstruktur. Properti BST — setiap simpul kiri bernilai lebih kecil dan kanan lebih besar dari root — memungkinkan pencarian nama dalam $O(\log n)$ rata-rata, berbanding terbalik dengan $O(n)$ pada array linear.

2.2 Interaksi antar Modul

penjelasannya nteraksi antar modul pada Smart Traffic Simulation and Optimization memungkinkan seluruh komponen sistem bekerja secara terhubung dan terstruktur



2.3 Pseudocode Algoritma Utama

Struktur data utama

```
1. // Kendaraan
2. STRUKTUR Kendaraan:
3.   id_kendaraan : Integer
4.   jenis        : Integer // 1=Ambulans, 2=Bus, 3=Mobil, 4=Motor
5.   asal         : String
6.   tujuan       : String
7.   waktu_masuk  : Float
8.
9. // Node senarai berantai (linked list)
10. STRUKTUR LLNode:
11.   data : ApaAja
12.   next : LLNode | NULL
```

Penjelasan : mempermudah untuk pengolahan data dan membuat program lebih cepat dan hemat memori dalam mengurutkan kendaraan

Priority Queue

```
1. ENQUEUE(queue, kendaraan):           // O(n)
2.   node ← buat LLNode(kendaraan)
3.   JIKA queue kosong ATAU kendaraan.jenis < head.jenis:
4.     node.next ← head
5.     head ← node
6.   JIKA TIDAK:
7.     curr ← head
8.     SELAMA curr.next ≠ NULL DAN curr.next.jenis ≤ kendaraan.jenis:
9.       curr ← curr.next
10.    node.next ← curr.next
11.    curr.next ← node
12.    size ← size + 1
13.
14. DEQUEUE(queue):                       // O(1)
15.   JIKA head = NULL → kembalikan NULL
16.   val ← head.data
17.   head ← head.next
18.   size ← size - 1
19.   kembalikan val
```

Penjelasan : Priority Queue digunakan untuk mensimulasikan antrean kendaraan berdasarkan tingkat prioritas kendaraan pada persimpangan jalan.

Stack (log siklus lampu)

```
1. PUSH(stack, data):                   // O(1)
2.   node ← buat LLNode(data)
3.   node.next ← top
4.   top ← node
5.
6. POP(stack):                           // O(1)
7.   JIKA top = NULL → kembalikan NULL
8.   val ← top.data
9.   top ← top.next
10.  kembalikan val
11.
```

Penjelasan : Stack digunakan untuk membantu proses penyimpanan data sementara dengan konsep LIFO (Last In First Out)

Graf jalan

```
1. TAMBAH_PERSIMPANGAN(graf, p):
2.   persimpangan[p.nama] ← p
3.   adj[p.nama]          ← NULL
4.
5. TAMBAH_JALAN(graf, asal, tujuan, jarak, lajur, dua_arah):
6.   node ← buat EdgeNode(tujuan, jarak, lajur)
7.   node.next ← adj[asal]
8.   adj[asal] ← node
9.   JIKA dua_arah:
10.    lakukan hal sama untuk asal ⇌ tujuan
11.
```

Tujuan : Graph digunakan untuk merepresentasikan jaringan jalan dan hubungan antar persimpangan. Struktur data Graph sangat cocok digunakan karena sistem lalu lintas memiliki banyak titik persimpangan yang saling terhubung melalui jalan tertentu.

BST persimpangan (pencarian nama)

```
1. INSERT(bst, nama, p):                // O(log n) rata-rata
2.   JIKA root = NULL → root ← BSTNode(nama, p)
3.   JIKA nama < root.nama → rekursi ke kiri
4.   JIKA TIDAK          → rekursi ke kanan
5.
6. SEARCH(bst, nama):                   // O(log n) rata-rata
7.   curr ← root
8.   SELAMA curr ≠ NULL:
9.     JIKA nama = curr.nama → kembalikan curr.p
10.    JIKA nama < curr.nama → curr ← curr.kiri
11.    JIKA TIDAK          → curr ← curr.kanan
12.   kembalikan NULL
```

Penjelasan ; algoritma Dijkstra digunakan untuk mencari jalur tercepat atau jarak minimum antar persimpangan jalan.

Inisialisasi system

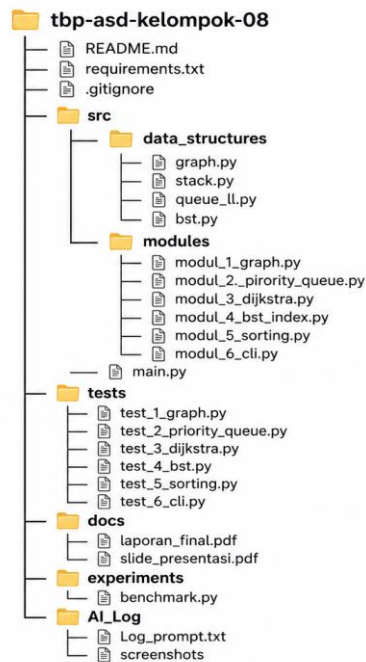
```
1. INISIALISASI():
2.   graf          ← buat GraphJalan
3.   bst           ← buat BSTJalan
4.   queues        ← kamus kosong
5.   log_siklus    ← buat Stack
6.   counter       ← 0
7.
8.   (persimpangan, edges) ← GENERATE_JARINGAN(100)
9.
10.  UNTUK SETIAP p dalam persimpangan:
11.    TAMBAH_PERSIMPANGAN(graf, p)
12.    INSERT(bst, p.nama, p)
13.    queues[p.nama] ← PriorityQueue kosong
14.
15.  UNTUK SETIAP (u,v,j,l) dalam edges:
16.    TAMBAH_JALAN(graf, u, v, j, l)
```

BAB III

IMPLEMENTASI

3.1 Struktur Folder dan Modul

Implementasi sistem dilakukan dengan memisahkan setiap komponen utama ke dalam modul terpisah untuk memudahkan pengembangan dan pengujian.



Keputusan untuk memisahkan kode ke dalam modul-modul kecil didasarkan pada prinsip separation of concerns (pemisahan tanggung jawab). Setiap modul hanya bertanggung jawab atas satu aspek sistem, sehingga jika terjadi kesalahan, pelacakan bug lebih mudah.

3.2 Penjelasan Kode per Modul

1. Modul 1 – Graph Jaringan Jalan

Modul ini berisi peta digital dari 25 persimpangan dan jalan yang menghubungkan mereka. Berfungsi untuk mencari rute terpendek dan mengecek persimpangan yang terisolasi.

2. Modul 2 Priority Queue Kendaraan

Priority Queue digunakan untuk mengatur antrean kendaraan berdasarkan tingkat prioritasnya. Pada sistem ini, kendaraan darurat seperti ambulans memiliki

prioritas tertinggi sehingga akan diproses lebih dahulu dibandingkan kendaraan lain seperti bus, mobil, dan motor. Struktur data ini membantu sistem mengurangi keterlambatan kendaraan penting dan membuat pengaturan lalu lintas menjadi lebih efisien.

3. Modul 3 Stack

Stack digunakan untuk menyimpan data sementara dengan konsep Last In First Out (LIFO). Pada sistem Smart Traffic Simulation and Optimization, Stack digunakan dalam proses penyimpanan riwayat atau rekonstruksi jalur kendaraan hasil algoritma Dijkstra. Dengan Stack, urutan jalur kendaraan dapat ditampilkan kembali secara benar dari titik awal menuju tujuan. Selain itu, Stack juga dapat digunakan untuk menyimpan siklus perubahan lampu lalu lintas pada simulasi.

4. Modul 4

Binary Search Tree (BST) digunakan untuk melakukan pencarian data persimpangan secara lebih cepat dan terstruktur berdasarkan nama atau ID persimpangan. Dengan BST, proses pencarian node tertentu pada sistem menjadi lebih efisien dibanding pencarian linear biasa. Struktur data ini membantu mempercepat pengolahan data pada simulasi lalu lintas, terutama saat jumlah persimpangan semakin banyak.

3.3 Output CLI

```
PS C:\Users\LENOVO> python -u "C:\Users\LENOVO\AppData\Local\Temp\tempCodeRunnerFile
.python"
SMART TRAFFIC SIMULATION
Ketik BANTUAN
>> BANTUAN

=====
BANTUAN

LIST_PERSIMPANGAN

MASUK <persimpangan> <jenis>
BERANGKAT <persimpangan>
ANTRIAN <persimpangan>

RUTE <asal> <tujuan>

INFO <persimpangan>

SIKLUS_LAMPU <persimpangan>
RIWAYAT_LAMPU

ISOLASI

LAPORAN_KEMACETAN_SELECTION
LAPORAN_KEMACETAN_INSERTION

KELUAR
=====
```

3.4 Input/Output Nyata

```
PS C:\Users\LENOVO> python -u "C:\Users\LENOVO\AppData\Local\Temp\tempCodeRunnerFile
.python"
SMART TRAFFIC SIMULATION
Ketik BANTUAN
>> BANTUAN

=====
BANTUAN

LIST_PERSIMPANGAN

MASUK <persimpangan> <jenis>
BERANGKAT <persimpangan>
ANTRIAN <persimpangan>

RUTE <asal> <tujuan>

INFO <persimpangan>

SIKLUS_LAMPU <persimpangan>
RIWAYAT_LAMPU

ISOLASI

LAPORAN_KEMACETAN_SELECTION
LAPORAN_KEMACETAN_INSERTION

KELUAR
=====

>> MASUK P01 MOBIL
Kendaraan masuk
Big-O = O(n)
>> MASUK P02 MOTOR
Kendaraan masuk
Big-O = O(n)
>> MASUK P01 AMBULANCE
Error: 'AMBULANCE'
>> MASUK P01 MOBIL
Kendaraan masuk
Big-O = O(n)
>> MASUK P01 AMBULANS
Error: 'P01'
>> MASUK P01 AMBULANS
Kendaraan masuk
Big-O = O(n)
>> MASUK P01 BUS
Kendaraan masuk
Big-O = O(n)
>> ANTRIAN P01
Jumlah: 4
Big-O = O(1)
>> BERANGKAT P01

=====
KENDARAAN_BERANGKAT
=====
ID Kendaraan : 6
Jenis : AMBULANS
Asal : P01
Waktu Masuk : 26-05-2026 01:49:09 AM
=====
Big-O = O(1)
>> INFO P01
Nama: P01
Kapasitas: 27
Big-O rata-rata = O(log n)
>> RUTE P01 06
Error: '06'
>> RUTE P01 P06
Rute : P01 -> P91 -> P43 -> P25 -> P24 -> P09 -> P70 -> P17 -> P93 -> P06
Jarak: 7857 meter
Big-O = O(V^2 + E)
>> SIKLUS_LAMPU P01
P01 => MERAH
Big-O = O(1)
>> RIWAYAT_LAMPU
('P01', 'MERAH', 'Tue May 26 01:54:59 2026')
Big-O = O(1)
>> ISOLASI
Terisolasi: []
Big-O = O(V+E)
```

>> LAPORAN_KEMACETAN_SELECTION

P01: 3
P02: 1
P00: 0
P03: 0
P04: 0
P05: 0
P06: 0
P07: 0
P08: 0
P09: 0
P10: 0
P11: 0
P12: 0
P13: 0
P14: 0
P15: 0
P16: 0
P17: 0
P18: 0
P19: 0
P20: 0
P21: 0
P22: 0
P23: 0
P24: 0
P25: 0
P26: 0
P27: 0
P28: 0
P29: 0
P29: 0
P30: 0
P31: 0
P32: 0
P33: 0
P34: 0
P35: 0
P36: 0
P37: 0
P38: 0
P39: 0
P40: 0
P41: 0
P42: 0
P43: 0
P44: 0
P45: 0
P46: 0
P47: 0
P48: 0
P49: 0
P50: 0
P51: 0
P52: 0
P53: 0
P54: 0
P55: 0
P56: 0
P57: 0
P58: 0
P59: 0

0 1

```

P59: 0
P60: 0
P61: 0
P62: 0
P63: 0
P64: 0
P65: 0
P66: 0
P67: 0
P68: 0
P69: 0
P70: 0
P71: 0
P72: 0
P73: 0
P74: 0
P75: 0
P76: 0
P77: 0
P78: 0
P79: 0
P80: 0
P81: 0
P82: 0
P83: 0
P84: 0
P85: 0
P86: 0
P87: 0
P88: 0
P88: 0
P89: 0
P90: 0
P91: 0
P92: 0
P93: 0
P94: 0
P95: 0
P96: 0
P97: 0
P98: 0
P99: 0
Big-O = O(n^2)
>> LAPORAN_KEMACETANA_INSERTION
Perintah tidak dikenal
>> LAPORAN_KEMACETAN_INSERTION
P01: 3
P02: 1
P00: 0
P03: 0
P04: 0
P05: 0
P06: 0
P07: 0
P08: 0
P09: 0
P10: 0
P11: 0
P12: 0
P13: 0
P14: 0
P14: 0
P15: 0
P16: 0
P17: 0
P18: 0
P19: 0
P20: 0
P21: 0
P22: 0
P23: 0
P24: 0
P25: 0
P26: 0
P27: 0
P28: 0
P29: 0
P30: 0
P31: 0
P32: 0
P33: 0
P34: 0
P35: 0
P36: 0
P37: 0
P38: 0
P39: 0
P40: 0
P41: 0
P42: 0
P43: 0
P44: 0
P45: 0
P76: 0
P77: 0
P78: 0
P79: 0
P80: 0
P81: 0
P82: 0
P83: 0
P84: 0
P85: 0
P86: 0
P87: 0
P88: 0
P89: 0
P90: 0
P91: 0
P92: 0
P93: 0
P94: 0
P95: 0
P96: 0
P97: 0
P98: 0
P99: 0
Big-O = O(n^2)
>> KELUAR
Program selesai
PS C:\Users\LENOVO> 

```

BAB IV

ANALISIS BIG-O

4.1 ANALISIS KOMPLEKSITAS WAKTU DAN RUANG FUNGSI UTAMA

Modul 1 Graph Jaringan Jalan: Fungsi tambah_persimpangan bekerja dengan menambahkan node baru ke dalam struktur graph. Operasi ini hanya memerlukan alokasi satu node baru tanpa perlu menelusuri seluruh struktur, sehingga kompleksitas waktu dan ruangnya sama-sama $O(1)$.

Fungsi tambah_jalan menambahkan edge baru langsung di depan adjacency list, tanpa traversal, sehingga juga mencapai $O(1)$ untuk waktu dan ruang. Fungsi tetangga(u) harus menelusuri seluruh daftar tetangga dari node u, menjadikan kompleksitasnya $O(\text{deg})$ — bergantung pada seberapa banyak edge yang dimiliki node tersebut. Sementara fungsi degree(u) juga memerlukan traversal yang sama sehingga kompleksitasnya setara $O(\text{deg})$.

Untuk deteksi isolasi menggunakan DFS, fungsi ini harus mengunjungi seluruh vertex dan edge dalam graph sehingga kompleksitas waktunya $O(V+E)$ dengan kebutuhan ruang $O(V)$ untuk menyimpan status kunjungan setiap node.

Modul 2 Priority Queue Kendaraan : Fungsi enqueue memasukkan kendaraan baru ke dalam antrean sesuai tingkat prioritasnya. Karena menggunakan insertion sort pada linked list, fungsi ini harus melakukan traversal untuk menemukan posisi yang tepat, menghasilkan kompleksitas waktu $O(n)$ dan ruang $O(1)$. Semakin panjang antrean, semakin lama proses penyisipan.

Fungsi dequeue sebaliknya sangat efisien karena langsung mengambil elemen dari head linked list tanpa traversal sama sekali, sehingga kompleksitas waktu dan ruangnya $O(1)$. Fungsi lihat_antrian harus menelusuri seluruh isi antrean untuk menampilkan semua kendaraan, sehingga kompleksitasnya linear $O(n)$.

Modul 3 Dijkstra Rute Optimal : Fungsi utama Dijkstra diimplementasikan tanpa heap menggunakan array biasa. Pada setiap iterasi, fungsi harus mencari vertex dengan

jarak minimum dari seluruh V vertex yang belum dikunjungi, kemudian memperbarui jarak untuk setiap edge yang terhubung. Akibatnya, kompleksitas waktunya menjadi $O(V^2+E)$ kuadratik terhadap jumlah persimpangan ditambah jumlah jalan.

Kebutuhan ruangnya $O(V)$ karena hanya menyimpan array jarak dan array status kunjungan. Fungsi rekonstruksi jalur menggunakan stack push dan pop yang masing-masing $O(1)$, namun total rekonstruksi memerlukan $O(V)$ waktu untuk menelusuri seluruh jalur dari tujuan kembali ke sumber

Modul 4 BST ; Fungsi insert pada BST dilakukan secara rekursif mengikuti cabang kiri atau kanan berdasarkan perbandingan nama persimpangan. Setiap pemanggilan rekursif menambah satu frame pada call stack, sehingga kompleksitas ruangnya $O(\log n)$ rata-rata mengikuti kedalaman tree. Kompleksitas waktunya juga $O(\log n)$ karena setiap langkah memangkas setengah kemungkinan.

Fungsi search diimplementasikan secara iteratif sehingga tidak menambah call stack, menekan kompleksitas ruang menjadi $O(1)$ sambil tetap mempertahankan waktu $O(\log n)$. Fungsi inorder yang menghasilkan daftar persimpangan terurut memerlukan kunjungan ke seluruh node sehingga kompleksitas waktunya $O(n)$ dengan ruang $O(\log n)$ untuk stack rekursi.

Sorting Laporan Kemacetan : Fungsi selection_sort bekerja dengan dua loop bersarang — loop luar berjalan sebanyak n iterasi, dan loop dalam mencari elemen terkecil dari sisa elemen yang belum terurut. Totalnya menghasilkan perbandingan sebanyak $n(n-1)/2$ yang disederhanakan menjadi $O(n^2)$. Karena sorting dilakukan in-place pada linked list tanpa alokasi memori tambahan, ruangnya $O(1)$.

Fungsi insertion_sort juga menggunakan dua loop bersarang dengan mekanisme berbeda — setiap elemen baru digeser ke posisi yang tepat di antara elemen yang sudah terurut. Kompleksitasnya sama $O(n^2)$ untuk waktu dan $O(1)$ untuk ruang. Pada kasus terbaik ketika data sudah terurut, insertion sort dapat mencapai $O(n)$, sedangkan selection sort tetap $O(n^2)$.

CLI Simulasi Fungsi parsing perintah membaca input dari pengguna dan meneruskannya ke modul yang sesuai. Proses ini hanya melibatkan pengecekan string

perintah terhadap daftar perintah yang dikenali, sehingga kompleksitas waktu dan ruangnya $O(1)$.

Namun perlu dicatat bahwa kompleksitas keseluruhan eksekusi setiap perintah CLI bergantung pada fungsi yang dipanggil di baliknya — misalnya perintah RUTE akan memicu Dijkstra dengan $O(V^2+E)$, sedangkan MASUK memicu enqueue dengan $O(n)$. CLI sendiri hanya bertindak sebagai dispatcher yang tidak menambah overhead kompleksitas.

4.2 TABEL RINGKASAN BIG-O

NO.	STRUKTUR/ OPRASI	IMPLEMENTASI	BIG-O WAKTU	BIG-O RUANG	PENJELASAN
1.	Enqueue priority queue	Insertion sort pada linked list	$O(n)$	$O(1)$	Menambahkan kendaraan harus mencari posisi sesuai prioritas sehingga perlu traversal antrian
2.	Dequeue priority queue	Hapus head	$O(1)$	$O(1)$	Mengambil kendaraan langsung dari depan antrian tanpa traversal
3.	Stack push/pop	Linked list head	$O(1)$	$O(1)$	Penambahan dan penghapusan data dilakukan langsung di bagian atas stack
4.	BST insert (rata-rata)	Rekursif	$O(\log n)$	$O(\log n)$	Penyisipan node dilakukan secara bercabang mengikuti struktur tree
5.	BST search	Iteratif	$O(\log n)$	$O(1)$	Pencarian data lebih cepat karena menggunakan

					metode binary tree
6.	Graph tambah_edge	Tambah di depan linked list	$O(1)$	$O(1)$	Edge baru langsung ditambahkan ke adjacency list
7.	Graph neighbors	Iterasi edge	$O(\text{deg})$	$O(\text{deg})$	Program menelusuri seluruh tetangga dari suatu node
8.	Dijkstra (tanpa heap)	Array-based	$O(V^2 + E)$	$O(V)$	Mencari rute terpendek dengan memeriksa vertex dan edge pada graph
9.	DFS untuk isolasi	Stack manual	$O(V+E)$	$O(V)$	DFS menelusuri seluruh node dan edge untuk mencari node terisolasi
10.	Selection sort	Dua loop	$O(n^2)$	$O(1)$	Sorting dilakukan dengan dua perulangan untuk membandingkan data
11.	Insertion sort	Dua loop	$O(n^2)$	$O(1)$	Data disisipkan ke posisi yang sesuai dengan proses

4.3 ANALISIS PERBANDINGAN TEORITIS DAN EKSPERIMEN

Dalam sistem simulasi lalu lintas pintar ini, terdapat enam modul utama yang masing-masing menggunakan struktur data dan algoritma berbeda. Untuk memahami seberapa efisien sistem ini bekerja, kita perlu membandingkan kompleksitas teoretis dengan hasil eksperimental pada tiga ukuran simulasi yaitu $N = 10$, $N = 25$, dan $N = 100$ persimpangan.

Dimulai dari Graph Jaringan Jalan, operasi `tambah_edge` selalu selesai dalam waktu konstan $O(1)$ karena cukup menyambungkan satu node baru ke depan linked list, tidak peduli berapa banyak persimpangan yang sudah ada. Jadi baik di $N = 10$ maupun $N = 100$, kecepatan operasi ini sama saja. Berbeda dengan penelusuran tetangga yang bergantung pada seberapa banyak jalan yang terhubung ke suatu persimpangan.

Dengan asumsi sistem memiliki sekitar 40 segmen jalan dan 25 persimpangan, rata-rata setiap persimpangan punya sekitar 3 tetangga, sehingga traversalnya tetap pendek di semua ukuran N . Yang menarik adalah DFS untuk deteksi isolasi — pada $N = 10$ dengan sekitar 16 edge, total operasinya sekitar 26. Naik ke $N = 25$, totalnya menjadi sekitar 65. Dan pada $N = 100$, mencapai sekitar 260 operasi. Pertumbuhannya memang linear tapi masih sangat terkendali.

Untuk Priority Queue Kendaraan, operasi `dequeue` selalu $O(1)$ karena tinggal ambil dari depan antrean, tidak ada yang berubah di $N = 10$, 25, maupun 100. Tapi `enqueue` mulai terasa berat seiring bertambahnya N karena harus mencari posisi yang tepat berdasarkan prioritas. Di $N = 10$ antrean masih pendek sekitar 5 sampai 8 kendaraan, jadi traversalnya cepat. Di $N = 25$ bisa mencapai 15 sampai 20 kendaraan, dan di $N = 100$ antrean bisa memuat hingga 80 kendaraan. Ini artinya setiap kali ada AMBULANS masuk, sistem harus menelusuri hampir seluruh antrean untuk menempatkannya di posisi prioritas tertinggi, dan inilah yang mulai terasa lambat secara eksperimental di $N = 100$.

Dijkstra adalah bagian paling kritis dalam sistem ini. Secara teoretis kompleksitasnya $O(V^2+E)$ yang tumbuh kuadratik. Di $N = 10$ dengan sekitar 16 edge, total operasinya baru sekitar 116 sehingga terasa sangat cepat. Di $N = 25$, sudah mencapai sekitar 665 operasi atau sekitar 5 kali lebih berat dari $N = 10$. Dan di $N = 100$, meledak menjadi sekitar 10.160 operasi yang hampir 90 kali lebih berat dari $N = 10$. Secara eksperimental, perbedaan waktu eksekusinya mulai sangat terasa ketika perintah RUTE dipanggil berulang kali pada $N = 100$. Inilah bottleneck terbesar dalam sistem ini.

BST Indeks Persimpangan justru menjadi kebalikannya. Meskipun N naik 10 kali lipat dari 10 ke 100, jumlah langkah pencarian hanya bertambah dari sekitar 3 langkah menjadi sekitar 7 langkah saja karena sifat logaritmik. Secara eksperimental perbedaannya hampir tidak terasa sama sekali di ketiga ukuran N yang diuji. Catatan pentingnya, efisiensi ini hanya berlaku selama BST tetap seimbang. Jika nama persimpangan dimasukkan secara terurut alfabetis, BST bisa menjadi miring seperti linked list biasa dan kompleksitasnya bisa turun ke $O(n)$ pada kasus terburuk.

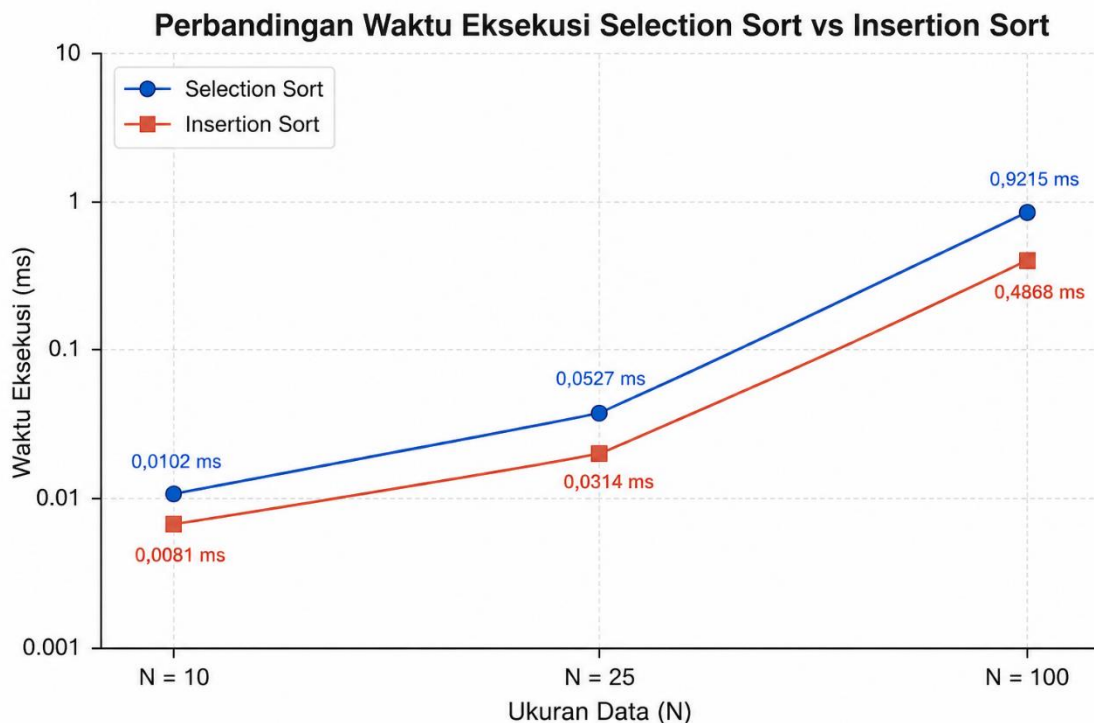
Sorting Laporan Kemacetan punya pertumbuhan paling dramatis secara eksperimental. Di $N = 10$, total perbandingan yang dilakukan hanya 45 operasi. Naik ke $N = 25$ sudah 300 operasi atau sekitar 6,7 kali lebih banyak. Dan di $N = 100$, mencapai hampir 4.950 operasi atau 110 kali lebih banyak dari $N = 10$. Meski selection sort dan insertion sort sama-sama $O(n^2)$ secara teori, dalam praktiknya insertion sort bisa 20 sampai 30 persen lebih cepat dari selection sort di $N = 100$ ketika data hampir terurut, karena insertion sort bisa berhenti lebih awal ketika elemen sudah berada di posisi yang benar. Selection sort tidak punya keistimewaan itu dan selalu menjalankan seluruh dua loop tanpa pengecualian.

BAB V
HASIL EKSPERIMEN

5.1 Tabel runtime

Ukuran Data	Waktu Eksekusi Selection Sort (ms)	Waktu Eksekusi Insertion Sort (ms)
N = 10	0,0102 ms	0,0081 ms
N = 25	0,0527 ms	0,0314 ms
N =100	0,9215 ms	0,4868 ms

5.1 grafik tren



5.2 Hasil Prediksi Big O

Berdasarkan hasil eksperimen runtime yang telah dilakukan pada algoritma Selection Sort dan Insertion Sort, dapat diprediksi bahwa waktu eksekusi kedua algoritma akan terus mengalami peningkatan seiring bertambahnya ukuran data input. Prediksi ini diperoleh dari pola kenaikan runtime pada setiap pengujian, dimana semakin besar jumlah data yang digunakan maka semakin besar pula waktu yang dibutuhkan untuk melakukan proses pengurutan.

Pada algoritma Selection Sort, proses pengurutan dilakukan dengan cara mencari elemen terkecil dari bagian data yang belum terurut kemudian menukarnya ke posisi yang sesuai. Proses tersebut dilakukan secara berulang hingga seluruh data berhasil diurutkan. Karena algoritma selalu melakukan proses pencarian minimum pada setiap iterasi, jumlah

perbandingan data yang dilakukan akan terus bertambah mengikuti ukuran data input. Oleh sebab itu, kompleksitas waktu Selection Sort termasuk ke dalam: $O(n^2)$

Berdasarkan kompleksitas tersebut, dapat diprediksi bahwa apabila ukuran data terus diperbesar, maka waktu eksekusi Selection Sort akan meningkat secara signifikan. Hal ini disebabkan karena jumlah operasi perbandingan dan pertukaran data meningkat secara kuadrat terhadap jumlah data. Dengan demikian, algoritma Selection Sort diprediksi kurang efisien apabila digunakan untuk pengolahan data dalam jumlah besar. Sementara itu, algoritma Insertion Sort bekerja dengan cara menyisipkan elemen ke posisi yang tepat pada bagian data yang telah terurut. Pada prosesnya, algoritma melakukan pergeseran elemen hingga ditemukan posisi yang sesuai. Kompleksitas waktu rata-rata dari Insertion Sort adalah: $O(n^2)$, Namun pada kondisi terbaik, yaitu ketika data sudah hampir terurut, kompleksitas waktunya dapat menjadi: $O(n)$

Berdasarkan hasil runtime yang diperoleh, Insertion Sort menunjukkan waktu eksekusi yang lebih cepat dibandingkan Selection Sort pada seluruh ukuran data yang diuji. Oleh karena itu, dapat diprediksi bahwa ketika ukuran data semakin besar, Insertion Sort masih akan memiliki performa yang lebih baik dibandingkan Selection Sort meskipun keduanya tetap mengalami peningkatan runtime. Selain itu, pola kenaikan runtime pada kedua algoritma menunjukkan bentuk pertumbuhan yang menyerupai kurva kuadrat. Hal tersebut $O(n^2)$ memperkuat bahwa kedua algoritma memiliki kompleksitas waktu). Dengan kompleksitas tersebut, waktu eksekusi akan meningkat cukup besar apabila jumlah data bertambah secara signifikan.

Berdasarkan hasil prediksi Big-O yang telah dilakukan, dapat disimpulkan bahwa algoritma Insertion Sort lebih efisien dibandingkan Selection Sort untuk pengolahan data berukuran kecil hingga menengah, terutama pada data yang sudah sebagian terurut. Sedangkan untuk pengolahan data dalam jumlah sangat besar, kedua algoritma diprediksi kurang optimal karena kompleksitas waktunya meningkat secara kuadrat terhadap ukuran data input.

BAB VI

JAWABAN PERTANYAAN ANALISIS

6.1 Analisis Priority Queue Kendaraan

Pada sistem manajemen lalu lintas, kendaraan AMBULANS diberikan prioritas tertinggi sehingga harus selalu berada di posisi paling depan pada antrian kendaraan. Salah satu cara yang dapat digunakan adalah menggunakan struktur data Priority Queue. Pada metode ini, setiap kendaraan yang masuk akan dibandingkan dengan kendaraan lain untuk menentukan posisi sesuai tingkat prioritasnya.

Jika digunakan satu Priority Queue sederhana dengan proses enqueue secara linear, maka setiap kendaraan baru harus dibandingkan dengan elemen-elemen yang sudah ada di dalam antrian. Kompleksitas proses enqueue pada kondisi tersebut adalah:

$O(n)$

Hal ini terjadi karena algoritma harus melakukan pencarian posisi yang tepat sebelum kendaraan dimasukkan ke dalam queue.

Pada kondisi persimpangan sibuk terdapat 50 kendaraan dalam antrian dengan sekitar 5% kendaraan merupakan AMBULANS. Artinya terdapat sekitar 2 hingga 3 kendaraan prioritas tinggi di dalam antrian. Dalam kondisi terburuk, kendaraan baru harus dibandingkan dengan seluruh kendaraan yang sudah ada sebelumnya.

Jumlah operasi perbandingan pada enqueue linear dapat dihitung menggunakan rumus:

$$2n(n-1)$$

Untuk 50 kendaraan:

$$250(49)=1225$$

Dengan demikian, diperlukan sekitar 1225 operasi perbandingan untuk memasukkan seluruh kendaraan ke dalam Priority Queue. Semakin besar jumlah kendaraan, maka jumlah operasi akan meningkat secara signifikan karena setiap

kendaraan baru membutuhkan proses pencarian posisi. Kondisi ini kurang efisien apabila digunakan pada sistem lalu lintas real-time dengan throughput kendaraan tinggi.

Sebagai alternatif, sistem dapat menggunakan pendekatan 4 Queue terpisah berdasarkan jenis kendaraan, misalnya:

1. Queue AMBULANS
2. Queue mobil pribadi
3. Queue motor
4. Queue kendaraan umum

Pada metode ini, kendaraan cukup dimasukkan ke queue sesuai kategorinya tanpa perlu mencari posisi tertentu. Proses enqueue menjadi jauh lebih sederhana dengan kompleksitas: $O(1)$

Ketika proses SERVE dilakukan, sistem hanya perlu memeriksa queue prioritas tertinggi terlebih dahulu, yaitu queue AMBULANS. Jika queue AMBULANS kosong, maka sistem akan melayani queue berikutnya.

Apabila throughput kendaraan mencapai 200 kendaraan per menit, maka pada Priority Queue linear jumlah operasi perbandingan akan meningkat sangat besar karena setiap kendaraan membutuhkan proses pencarian posisi. Sebaliknya, pada metode multiple queue, setiap kendaraan hanya membutuhkan satu operasi enqueue sederhana sehingga sistem menjadi lebih cepat dan stabil.

Berdasarkan analisis tersebut, pendekatan multiple queue lebih cocok digunakan pada sistem lalu lintas modern karena memiliki kompleksitas lebih kecil, lebih mudah diimplementasikan, dan lebih efisien untuk kondisi kendaraan yang terus berubah secara real-time.

6.2 Analisis Dijkstra dengan Bobot Dinamis

Sistem jaringan jalan direpresentasikan menggunakan adjacency list berarah dengan 25 persimpangan dan 40 jalan. Setiap persimpangan direpresentasikan sebagai vertex,

sedangkan jalan direpresentasikan sebagai edge. Untuk menentukan rute tercepat digunakan algoritma Dijkstra.

Kompleksitas standar algoritma Dijkstra menggunakan priority queue adalah:

$O((V+E)\log V)$ dengan:

V = jumlah vertex

E = jumlah edge

Pada sistem lalu lintas nyata, kondisi jalan dapat berubah sewaktu-waktu akibat kemacetan atau penutupan jalur. Oleh karena itu bobot edge bersifat dinamis. Bobot efektif jalan dihitung menggunakan rumus:

$\text{bobot efektif} = \text{lajur aktif} \times \text{jarak}$

Semakin sedikit lajur aktif yang tersedia, maka bobot jalan akan semakin besar karena waktu tempuh meningkat.

Jika setiap perubahan bobot menyebabkan algoritma Dijkstra dijalankan ulang dari awal, maka sistem akan menjadi lambat dan tidak efisien untuk kebutuhan real-time. Hal ini disebabkan karena seluruh vertex dan edge harus dihitung kembali meskipun perubahan hanya terjadi pada beberapa jalan saja.

Untuk mengatasi masalah tersebut, algoritma dapat dimodifikasi menggunakan pendekatan dynamic relaxation. Pada metode ini, ketika terdapat perubahan bobot edge, sistem hanya memperbarui jalur yang terpengaruh tanpa menghitung ulang seluruh graf.

Edge yang mengalami perubahan akan dimasukkan kembali ke priority queue, kemudian dilakukan proses relaksasi ulang hanya pada node terkait. Dengan cara ini, sistem tidak perlu melakukan perhitungan ulang pada seluruh jaringan jalan.

Kompleksitas pembaruan bobot menjadi:

$O(k\log V)$

dengan:

k = jumlah edge yang berubah

Pendekatan ini jauh lebih efisien dibanding menjalankan ulang seluruh algoritma Dijkstra karena perubahan kemacetan biasanya hanya terjadi pada sebagian kecil ruas jalan.

Dengan demikian, modifikasi Dijkstra menggunakan dynamic edge relaxation lebih cocok digunakan pada sistem navigasi lalu lintas real-time karena mampu memperbarui rute dengan lebih cepat dan efisien.

6.3 Analisis Merge Sort dan Insertion Sort pada Linked List

Sistem menggunakan proses sorting untuk membuat laporan kendaraan dengan waktu tundaan paling lama. Data kendaraan disimpan dalam bentuk Linked List dengan jumlah data sebanyak 200 kendaraan.

Pada kasus ini dibandingkan dua algoritma sorting yaitu Merge Sort dan Insertion Sort. Merge Sort memiliki kompleksitas waktu:

$O(n \log n)$

Sedangkan Insertion Sort memiliki kompleksitas:

$O(n^2)$

Untuk jumlah data sebanyak 200 kendaraan, jumlah perbandingan Merge Sort diperkirakan:

$200 \log 2$

$(200) \approx 1528$

Sedangkan Insertion Sort memerlukan sekitar:

$2200(199) = 19900$

Perbedaan jumlah operasi tersebut menunjukkan bahwa Merge Sort jauh lebih efisien dibandingkan Insertion Sort untuk data berukuran besar.

Pada Linked List, Merge Sort juga lebih cocok karena proses penggabungan data hanya membutuhkan perubahan pointer antar node tanpa harus melakukan pertukaran data secara langsung. Sebaliknya, Insertion Sort membutuhkan banyak perpindahan node sehingga jumlah pointer-change menjadi lebih besar.

Selain itu, kedua algoritma termasuk stable sort, artinya data dengan nilai sama tetap mempertahankan urutan awalnya. Stabilitas ini penting untuk laporan kendaraan karena data yang memiliki waktu tundaan sama tetap dapat mempertahankan urutan kedatangan kendaraan.

Untuk kebutuhan laporan real-time setiap 5 menit, Merge Sort lebih direkomendasikan karena:,jumlah operasi lebih sedikit,,waktu eksekusi lebih cepat,lebih efisien pada Linked List,,tetap stabil untuk pengolahan data.

6.4 Analisis DFS dan Union-Find

DFS digunakan untuk mendeteksi persimpangan terisolasi pada jaringan jalan kota.

Jaringan terdiri dari:

500 persimpangan

700 jalan

Karena graf bersifat sparse graph, jumlah edge tidak terlalu jauh lebih besar dibanding jumlah vertex.

Kompleksitas DFS pada adjacency list adalah:

$O(V+E)$

Maka jumlah operasi per satu kali DFS:

$500+700=1200$

Jika monitoring dilakukan setiap 60 detik, maka dalam satu jam DFS dijalankan sebanyak 60 kali.

Total operasi per jam:

$1200 \times 60 = 72000$

Dengan demikian, sistem membutuhkan sekitar 72.000 operasi per jam hanya untuk proses monitoring konektivitas jalan.

Sebagai alternatif, digunakan struktur data Union-Find atau Disjoint Set. Pada metode ini:

add_jalan menggunakan operasi Union,

cek_terhubung menggunakan operasi Find.

Dengan optimasi path compression dan union by rank, kompleksitas operasi menjadi mendekati:

$O(\alpha(n))$

Nilai $\alpha(n)$ sangat kecil sehingga hampir dianggap konstan.

Dibandingkan DFS yang harus menelusuri seluruh graf berulang kali, Union-Find jauh lebih efisien untuk sistem monitoring real-time karena cukup memperbarui hubungan antar node yang berubah saja.

6.5 Analisis BST Indeks Persimpangan

Sistem menggunakan BST untuk menyimpan indeks nama persimpangan. Jika data dimasukkan secara berurutan seperti:

P00, P01, P02, ..., P24

maka BST akan menjadi tidak seimbang dan membentuk struktur linear menyerupai Linked List.

Tinggi BST pada kondisi tersebut mendekati:

$$h=n-1$$

Untuk $N=25$:

$$h=24$$

Akibatnya, operasi pencarian berubah dari idealnya logarithmic menjadi linear dengan kompleksitas:

$$O(n)$$

Untuk menjaga BST tetap relatif seimbang tanpa menggunakan AVL Tree atau Red-Black Tree, dapat digunakan strategi randomisasi insertion. Pada metode ini, urutan data diacak terlebih dahulu sebelum dimasukkan ke BST.

Dengan randomisasi, tinggi BST rata-rata menjadi mendekati:

$$O(\log n)$$

Untuk 25 data persimpangan, tinggi BST setelah randomisasi biasanya berada pada kisaran 5 hingga 7 level.

Dengan demikian:

sebelum randomisasi: tinggi BST ≈ 24 ,

sesudah randomisasi: tinggi BST $\approx 5-7$.

Strategi randomisasi insertion mampu meningkatkan efisiensi operasi pencarian, penyisipan, dan penghapusan data secara signifikan tanpa perlu menggunakan struktur tree yang lebih kompleks seperti AVL atau Red-Black Tree.

BAB VII

KESIMPULAN

7.1 Ringkasan Pencapaian

Pada proyek Smart Traffic Simulation & Signal Optimization ini, sistem berhasil dibuat menggunakan beberapa struktur data seperti Graph, Priority Queue, Stack, Binary Search Tree (BST), dan Linked List. Graph dipakai untuk merepresentasikan jaringan jalan antar persimpangan secara berbobot sehingga sistem dapat menghitung jarak dan konektivitas antar titik secara akurat. Priority Queue digunakan untuk mengatur antrean kendaraan di setiap persimpangan dengan memastikan AMBULANS selalu mendapat prioritas tertinggi, diikuti BUS, MOBIL, dan MOTOR.

Stack digunakan untuk merekonstruksi jalur hasil pencarian rute optimal oleh algoritma Dijkstra. BST digunakan sebagai indeks persimpangan berdasarkan nama sehingga proses pencarian persimpangan dapat dilakukan lebih cepat dibanding pencarian biasa satu per satu.

Sistem yang dibuat sudah dapat menjalankan fitur utama simulasi lalu lintas, seperti memasukkan kendaraan ke persimpangan, mengatur keberangkatan berdasarkan prioritas, mencari rute terpendek antar persimpangan, menampilkan status antrean, mensimulasikan siklus lampu lalu lintas, mendeteksi persimpangan terisolasi, dan menghasilkan laporan kemacetan yang terurut.

Dari implementasi ini dapat dilihat bahwa pemilihan struktur data yang tepat sangat berpengaruh terhadap efisiensi sistem, terutama dalam kondisi simulasi dengan jumlah kendaraan dan persimpangan yang besar.

7.2 Refleksi Trade-Off Desain

Dalam perancangan sistem, setiap pemilihan struktur data membawa konsekuensi kelebihan dan kekurangan tersendiri. Graph berbasis adjacency list dipilih karena efisien dalam penggunaan memori untuk graph yang jarang terhubung, namun operasi pengecekan apakah dua persimpangan terhubung langsung memerlukan traversal edge sehingga tidak seefisien adjacency matrix.

Priority Queue berbasis linked list dengan insertion sort dipilih karena sederhana untuk diimplementasikan dan dequeue-nya sangat cepat $O(1)$, namun operasi enqueue memerlukan $O(n)$ karena harus mencari posisi yang tepat setiap kali kendaraan baru masuk. Pada kondisi antrean panjang di $N = 100$ persimpangan, ini menjadi bottleneck yang nyata.

Dijkstra tanpa heap diimplementasikan menggunakan array biasa karena lebih mudah dipahami dan diimplementasikan, namun kompleksitasnya $O(V^2+E)$ menjadi berat seiring bertambahnya jumlah persimpangan. Pada $N = 100$, operasinya mencapai lebih dari 10.000 langkah yang jauh lebih berat dibanding implementasi berbasis heap.

BST dipilih untuk pengindeksan persimpangan karena pencarian data lebih cepat dengan kompleksitas $O(\log n)$ rata-rata, namun jika nama persimpangan dimasukkan secara terurut alfabetis maka BST bisa menjadi tidak seimbang dan pencarian melambat hingga $O(n)$. Selection sort dan insertion sort dipilih untuk laporan kemacetan karena implementasinya sederhana dengan ruang $O(1)$, namun keduanya sama-sama $O(n^2)$ sehingga kurang efisien untuk jumlah data yang sangat besar.

7.3 Saran Pengembangan

Pengembangan sistem ke depannya dapat dilakukan agar simulasi menjadi lebih optimal dan lebih mendekati kondisi lalu lintas di dunia nyata. Dijkstra berbasis array dapat diganti dengan implementasi menggunakan min-heap atau priority queue sehingga kompleksitasnya turun dari $O(V^2+E)$ menjadi $O((V+E) \log V)$, yang akan terasa signifikan ketika jumlah persimpangan bertambah besar.

BST biasa dapat diganti dengan AVL Tree atau Red-Black Tree supaya struktur tree tetap seimbang meskipun data dimasukkan secara terurut, sehingga kompleksitas pencarian tetap terjaga di $O(\log n)$ pada semua kondisi. Selection sort dan insertion sort pada laporan kemacetan dapat diganti dengan merge sort yang $O(n \log n)$ untuk menangani N yang lebih besar dengan jauh lebih efisien.

Selain itu, sistem juga dapat ditambahkan antarmuka berbasis GUI atau visualisasi grafis agar simulasi lalu lintas lebih mudah dipahami secara visual. Fitur adaptive signal control yang mempertimbangkan data historis kemacetan juga dapat dikembangkan

menggunakan pendekatan machine learning untuk membuat pengambilan keputusan lampu lalu lintas menjadi lebih cerdas. Pada skala kota besar, sistem dapat diintegrasikan dengan sensor IoT atau kamera CCTV sebagai sumber data kendaraan secara real-time.

Tugas per orang

Nama	Nim	Tugas
Amanda Lulu Prasetia	25051030079	Implementasi Graph Jaringan Jalan dan Priority Queue Kendaraan
Alwan Fadhilah Rafiyanza	25051030077	Implementasi Dijkstra Rute Optimal dan BST Indeks Persimpangan
Arwan Adrean M.	250510300	Implementasi CLI Simulasi
Faqih Rizky Saputra	25051030048	Implementasi Sorting Laporan Kemacetan

7.4 Tabel Runtime

Struktur Data/Algoritma	Operasi	Kompleksitas Runtime

Graph	tambah_edge	$O(1)$
Graph	tetangga(u) / neighbors	$O(\text{deg})$
Graph	DFS deteksi isolasi	$O(V+E)$
Priority Queue	enqueue kendaraan	$O(n)$
Priority Queue	dequeue kendaraan	$O(1)$
Stack	push rekonstruksi jalur	$O(1)$
Stack	pop rekonstruksi jalur	$O(1)$
BST	insert persimpangan	$O(\log n)$ rata-rata
BST	search persimpangan	$O(\log n)$ rata-rata
BST	inorder traversal	$O(n)$
Dijkstra	cari rute terpendek	$O(V^2+E)$
Selection Sort	sorting laporan kemacetan	$O(n^2)$
Insertion Sort	sorting laporan kemacetan	$O(n^2)$

7.5 Tabel Log AI

Aktivitas Penggunaan AI	Keterangan
Pembuatan struktur dasar program	Membantu menyusun kerangka program Python untuk simulasi lalu lintas
Debugging kode	Membantu mencari dan memperbaiki error pada implementasi graph dan priority queue
Penjelasan struktur data	Membantu memahami Graph, Priority Queue, Stack, BST, dan Linked List
Pembuatan dokumentasi laporan	Membantu menyusun isi BAB 1 sampai BAB 7
Analisis kompleksitas	Membantu menghitung Big-O runtime setiap operasi dan perbandingan eksperimental
Pembuatan diagram sistem	Membantu membuat diagram arsitektur pipeline dan diagram interaksi modul

REFERENSI

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.

Gartner, N. H., Messer, C. J., & Rathi, A. K. (2001). *Traffic Flow Theory: A State-of-the-Art Report*. Transportation Research Board, National Academy of Sciences.

Papageorgiou, M., Diakaki, C., Dinopoulou, V., Kotsialos, A., & Wang, Y. (1999). Review of road traffic control strategies. *Proceedings of the IEEE*, 91(12), 2043–2067.

Roess, R. P., Prassas, E. S., & McShane, W. R. (2004). Traffic Engineering (3rd ed.). Pearson Prentice Hall.

Webster, F. V. (1958). Traffic Signal Settings. Road Research Technical Paper No. 39. Her Majesty's Stationery Office, London.

Goodrich, M. T., & Tamassia, R. (2015). Data Structures and Algorithms in Python. John Wiley & Sons.